

Rossana Palma López
Segunda Tarea - Examen Complejidad Computacional

1.-

a) testigo w es una coloración c tal que a cada vertice v en V de una grafica G se le asigna un color diferente $(v, c(v))$ en el caso de 3-COL. $V \rightarrow \{1, 2, 3\}$

b) ya que el testigo debe de estar en función de $B_{in}(G)$ representación binaria de la grafica G , entonces usaremos bits para representar los vertices V las aristas E y su coloración correspondiente c de todo v en V

Cada vertice es un número del 1 a n (para 3-col. 1 a 3)
la representación de este número necesita $\log_2 n$ bits por lo que toma $O(\log_2 n)$ pasos escribiendo

Al tener n vertices entonces escribir todos los numeros toma $n \cdot O(\log_2 n) = O(n \log_2 n)$

Cada color en este caso nos toma 1 paso escribiendo entonces el testigo toma $O(n \log_2 n)$ pasos por lo que es de tamaño polinomial en $|B_{in}(G)|$

c) Un verificado para 3-COL quedaria de la siguiente forma

recibe una grafica G y una coloración c .
comprueba que c es una función que se cumple para los vertices V de G y que ninguna arista en E de G tiene ambos extremos u y v en V de G del mismo color, es decir usando la definición de una 3-COL.
Entonces el verificador primero recorre todos los vertices V en G , para cada v en V verifica que $c(v)$ existe y esta en $\{1, 2, 3\}$. En caso de que algún vertice no tenga

color o existe un color fuera de 1, 2, 3 rechazamos,

Los pasos necesarios para realizar este recorrido son $|V|$ y $|V| = n$ así que toma $O(n)$.

Después el verificador recorre las aristas E de G . para cada arista uv en E revisa si $c(u)$ y $c(v)$ son iguales de ser así rechaza.

Los pasos necesarios para realizar este recorrido son $|E|$ y $|E| = m$ ya que revisar $c(v)$ toma $O(1)$ así que $m \times O(1) = O(m)$.

Si el verificador no rechaza en ningún momento al testigo w entonces c es una 3-COL válida de G y aceptamos. El verificador toma $O(n) + O(m)$ pasos $O(n+m)$ es polinomial.

d) 3COL $\in$ NP ya que

1) Tiene un testigo $w = c: V \rightarrow \{1, 2, 3\}$

2) $|w| = O(n \log_2 n)$ es polinomial

3) El verificador toma tiempo $O(n+m)$ que es polinomial

4) Dado que por 1), 2) y 3) $G \in$ 3COL

$\exists c$ tal que verificador 3COL($B_m(G), c$) = 1 pues lo que si existe 3-COL el verificador la acepta en lo que c es una coloración válida

$\therefore$ 3-COL $\in$ NP

e) 3COL esta en P

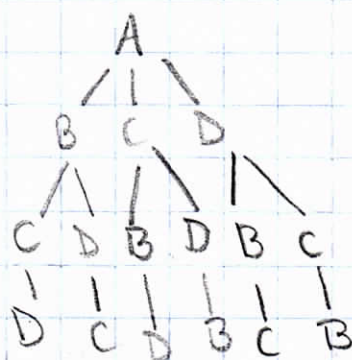
Por lo visto en clase si 3-COL $\in$ NP para estar en P tendríamos poderse reducir a 3-SAT Si

esto fuera posible $NP=P$ lo cual no ha sido demostrado

2.-

2a)

Implementemos el problema del viajero para 4 ciudades. A, B, C, D, si queremos visitar todas una vez y regresar al inicio, es decir un ciclo hamiltoniano, tomemos a A como la ciudad de partida, desde A podemos visitar B, C u D, cada que visitemos una ciudad las posibilidades se reducen en uno, así que podemos verlo como un árbol de decisión que luce de la siguiente manera



lo cual resulta en 6 caminos diferentes

el primer nivel tiene 3 ramas
el segundo nivel tiene 2 ramas
el ultimo nivel tiene 1 ramas
 $3 \times 2 \times 1 = 6$

así que $n-1$ opciones para la primera ciudad a visitar, $n-2$ para la segunda y $n-3$ para la tercera, esto para $n=4$ es $(n-1)! = (4-1)!$
 $(3)! = 6$

Al ser factorial nos alejamos bastante en que sea polinomial así que $TSP \in P$

2b) $TSP^* \in P$

TSP^* consta de G, P, B como parametros y P es la ruta candidata, así que funciona como testimonio de TSP .

i) verificar que P visita cada vértice una vez, es decir que cumple con la definición de ser un ciclo hamiltoniano.

El verificador debe recorrer la ruta P y marcar visitado cada vértice V de G . usamos un arreglo booleano de los vértices V de G que inicializamos en *false*, por cada vértice V de P marcamos *true* en el arreglo booleano de vértices V de G , eso quiere decir que el vértice ha sido visitado, así que si al recorrer P encontramos en el arreglo un *true* rechazamos por que el vértice ya habria sido visitado.

Al finalizar el arreglo debe ser *true* por cada vértice V de G . Toma $|V|$ pasos y $|V|=n, O(n)$

ii) verificar que cada arista de P existe en G , es decir que por cada par de vértices consecutivos en P buscar la arista formada por ese par de vértices consecutivos en la matriz de adyacencia de G . Si no existe rechazamos. P tiene n aristas ya que es un ciclo de n vértices. Cada consulta es a lo mas la cantidad de vértices puesto que el grado de cada vértice es menor o igual al los vértices V de G totales, en el peor caso toma n^2 verificar cada arista. $O(n^2)$

iii) calcular $\text{cost}(P)$ y verificar $\text{cost}(P) \leq B$, es decir que si el costo es mayor que B no es un testimonio válido para TSP, para verificar que $\text{cost}(P)$ no es mayor que B inicializamos un contador en 0, por cada arista formada por un par de vértices consecutivos en P , aumentamos el contador con el valor del peso de la arista en G . Al final si el contador es mayor a B rechazamos en caso contrario aceptamos, este proceso toma $|E|$ en G pesos $|E| = n$, $O(n)$

iv) La complejidad es polinomial ya que cada proceso del verificador toma tiempo polinomial

$$O(n) + O(n^2) + O(n)$$

Dado que $n \in |(G, P, B)|$ el tiempo que toma es polinomial respecto al tamaño de la entrada

$$\therefore \text{TSP}^* \in P$$

3.-

3a)

S es testimonio para VERTEXCOVER $\forall a$ que es un subconjunto de los vertices y precisamente necesitamos verificar que un vertex cover de tamaño K cubre a todo G y $|S| \leq K$

3b)

Si $K = |V|$ y $|V| = n$ entonces en el peor de los casos $K = n$, en función de $\text{Bin}(n)$ la representación binaria de los vertices V de G necesita $\log_2 n$ bits, entonces escribir todos los vertices toma $n \log_2 n$ pasos, $O(n \log_2 n)$, así que toma tiempo polinomial escribir el testigo elegido para VERTEXCOVER

3c) El verificador para VERTEXCOVER quedaria de la siguiente forma

S recibir una grafica G , un número K y el testimonio comprobamos que $|S| \leq K$ por lo que debemos recorrer el conjunto S y con un contador i aumentando por cada vertice en S si el contador es mayor a K rechazamos esto toma $|S|$ pasos y si $|S| = n$ $O(n)$

Verificamos después que $S \subseteq V$ es decir que recorremos S y por cada vértice en S revisamos si existe en los vértices V de G , esto toma $|S|$ y $|S| = n$, $O(n)$

Por último verificamos que S cubra todas las aristas de G recorremos E de G y revisamos que uv , donde uv está en E de G , estén en S , rechazamos si u o v no están en S , este proceso toma $|S|$ por cada arista en E , pasos, si $|S| = n$ y $|E| = m$, $O(n \cdot m)$

Si el verificador no rechaza para al testigo en ningún paso aceptamos y en total toma $O(n) + O(n) + O(n \cdot m)$

Como $(G, k) \in \text{VERTEXCOVER}$ existe un S testimonio tal que un verificador de VERTEXCOVER con G, k y S como parámetros da 1

3 d) $\text{VERTEXCOVER} \in \text{NP}$ ya que

1) Tiene un testigo $w = S$

2) $|w| = |S| \leq k$ con $k = |V|$ lo que es $O(\log_2 n)$ polinomial

3) El verificador toma tiempo $O(n) + O(n) + O(n \cdot m)$

4) Por 1), 2), 3) G es un VERTEXCOVER

Es tal que verificador $\text{VERTEXCOVER}(G, k, S) = 1$
 puesto que existe un VERTEXCOVER el verificador lo
 acepta entonces S cubre todas las aristas del
 VERTEXCOVER
 $\therefore \text{VERTEXCOVER} \in \text{NP}$

4a) S es testigo para MAXCUT de tamaño k para
 una gráfica G ya que $|S|$ es la cantidad de vertices
 uv que definen al corte compuesto por aristas cruzadas

4b) Si $|V| = n$ y en el peor de los casos el tamaño
 del MAXCUT es $|V|$ entonces $|V| = k$, en función de
 de $\text{Bin}(G)$ la representación binaria de los vertices V de G
 necesita $\log_2 n$ bits, entonces escribir todos los vertices
 toma $n \log_2 n$ pasos, $O(n \log_2 n)$ así que toma tiempo polinomial
 escribir el testigo elegido para MAXCUT

4c) El verificador para MAXCUT quedaría de la siguiente
 forma

Recibe una gráfica G , un número k y el testigo S
 comprobamos que $S \subseteq V$ es decir que recorremos S
 y por cada vertice en S revisamos si existe en los
 vertices V de G esto toma $|S|$ pasos y $|S| = n$,
 $O(n)$

verificamos que $|S| \geq k$ por lo que debemos
 recorrer el conjunto S y con un contador vr aumentando

por cada vertice en S si el contador es menor q K rechazamos esto toma $|S|$ pasos y si $|S|=n$, $O(n)$

Por ultimo verificamos que S cubra todas las aristas de G , recorremos E de G y revisamos que $u \in S$ y $v \in S$ o bien que $u \notin S$ y $v \notin S$, incrementamos un contador si lo anterior ocurre, rechazamos si u y $v \in S$ o u y $v \notin S$, este proceso toma $|S|$ por cada arista en E pasos, si $|S|=n$ y $|E|=m$, $O(n \cdot m)$. Si el verificador no rechaza para el testigo en ningun paso aceptamos y en total toma $O(n) + O(n) + O(n \cdot m)$

Como $(G, K) \in \text{MAXCUT}$ existe un S testigo tal que un verificador de MAXCUT con G, K, S como parametros da 1

4d) MAXCUT $\in$ NP ya que

1) Tiene un testigo $w=S$

2) $|w| = |S| \geq K$ con $K = |V|$ lo que es $O(\log_2 n)$ polinomial

3) El verificador toma tiempo $O(n) + O(n) + O(n \cdot m)$

4) Por 2), 2) y 3) G es MAXCUT

$\exists S$ tal que verificador MAXCUT $(G, K, S) = 1$ puesto que existe un MAXCUT el verificador lo acepta entonces S cubre todas las aristas del MAXCUT

$\therefore \text{MAXCUT} \in \text{NP}$

5.-

5a)

$L \in P$ entonces $L \in NP$

Si A es un algoritmo que muestra que $x \in L$, entonces basta con correr el algoritmo para saber si acepta o rechaza, el testigo no es importante porque se conoce el algoritmo que resuelve el problema aun con el testigo ya que no lo ocupa para saber si aceptar o rechazar.

$\exists$ El verificador corre en tiempo polinomial ya que el testigo ya no aporta tiempo adicional al algoritmo A que acepta que $L \in P$

$\exists$ El testigo puede ser incluso una cadena vacía así que es polinomial

y $x \in L$ si y solo si existe un testigo tal que un verificador que recibe x y el testigo es igual a 1 esto se cumple dado que verificador de L es igual al algoritmo con los parámetros x o testigo

Al pasar los tres pasos anteriores $L \in NP$.

5b)

Sea $coNP$ la clase de los complementos de los lenguajes en NP , entonces $L \in coNP$ significa que $L^c \in NP$

L^c está formado por todos los x que no están en L . Por el inciso anterior sabemos que si existe un algoritmo A que resuelve si $x \in L$ entonces un algoritmo que lo hace para L^c tendría que aceptar si $x \notin L$ y rechazar si $x \in L$.

Hacer esto toma solo un paso que es la negación a toda salida que genere A , lo cual es $O(1)$.

Al tomar tiempo polinomial A el algoritmo que decide L^c toma igualmente tiempo polinomial, al comprobar las mismas propiedades del inciso a) con L^c obtenemos lo mismo ya que el algoritmo solo niega la salida así que $L^c \in NP$ por lo que implica que $L \in coNP$.

5c).

Sea $L \in P$. Por el inciso a) $L \in NP$ y por b) $L \in coNP$ entonces $L \in NP \cap coNP$ entonces P está en la intersección de $NP \cap coNP$.

$$P \subseteq NP \cap coNP$$

6.-

Si $P = NP$ todo problema que sea verificable en tiempo polinomial se resolvería en tiempo polinomial también. $NP = coNP$ entonces $NP \subseteq coNP$ y $coNP \subseteq NP$.

a) $NP \subseteq coNP$

Sea un lenguaje $L \in NP$, sabemos que $L \in coNP$ si $L^c \in NP$. Dado que $P = NP$, entonces todo lenguaje en NP tendría que estar en P .

Por el ejercicio 5 vimos que $L^c \in P$ por que tenemos un algoritmo A que resuelve si $x \in L$ que al negarlo tenemos que resuelve si $x \notin L$ entonces $x \in L^c$, $L \in \text{coNP}$, como sabemos que $L \in \text{NP}$ entonces $\text{NP} \subseteq \text{coNP}$

b) $\text{coNP} \subseteq \text{NP}$

Si $L \in \text{coNP}$ entonces $L^c \in \text{NP}$ por definición, dado que $P = \text{NP}$, todo lenguaje $L \in \text{NP}$ y este $L \in P$ como $L \in \text{NP}$ entonces $L^c \in \text{NP}$ entonces $L \in P$ tambien, como tenemos el algoritmo A que resuelve si $x \in L^c$ y al negarlo resolvemos que $x \in L$ entonces $L \in P$, por el ejercicio 5 todo $L \in P$ tambien $L \in \text{NP}$, un verificador de L que toma A aceptara sin necesidad del testigo, entonces $L \in \text{NP}$, por lo tanto al ser L cualquier lenguaje en coNP y vimos que $L \in \text{NP}$ entonces $\text{coNP} \subseteq \text{NP}$

c) $\text{NP} = \text{coNP}$

Por a) tenemos que $\text{NP} \subseteq \text{coNP}$ y por b) que $\text{coNP} \subseteq \text{NP}$ por lo tanto $\text{NP} = \text{coNP}$